

WEEKLY ASSESSMENT - 7

1. Define a class Car with attributes and a method to display details.

A class wraps data parameters and structural execution actions together into an object template blueprint:

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def display_details(self):
        return f"Car: {self.brand} {self.model}"

my_car = Car("Toyota", "Corolla")
print(my_car.display_details()) # Output: Car: Toyota Corolla
```

2. Explain the purpose of the `__init__` method.

The `__init__` method acts as the reserved **constructor routine** in Python classes. It runs automatically when a new class instance is initialized, allocating structural memory values and initializing default state properties directly onto the newborn object instance using the `self` variable reference.

3. Create a subclass ElectricCar that inherits from Car.

Inheritance lets child sub-classes capture and reuse properties declared inside parent base classes seamlessly by using the `super()` routine setup:

```
class ElectricCar(Car):
    def __init__(self, brand, model, battery_capacity):
        super().__init__(brand, model)
        self.battery_capacity = battery_capacity

ev = ElectricCar("Tesla", "Model 3", "75 kWh")
print(ev.display_details()) # Output: Car: Tesla Model 3
```

4. Demonstrate "Method Overriding" in inheritance.

Method Overriding happens when a child subclass redefines a method name inherited from its parent to customize or alter its functionality:

```
class ElectricCar(Car):
    def __init__(self, brand, model, battery_capacity):
        super().__init__(brand, model)
        self.battery_capacity = battery_capacity
```

```

    # Overriding the parent display_details method
    def display_details(self):
        return f"Electric Car: {self.brand} {self.model} with a {self.battery_capacity} battery."

ev = ElectricCar("Tesla", "Model 3", "75 kWh")
print(ev.display_details()) # Output: Electric Car: Tesla Model 3 with a 75 kWh battery.

```

5. Explain "Encapsulation" by creating private attributes using double underscores.

Encapsulation keeps internal object data hidden from accidental external changes by locking attributes down using a double underscore prefix (`__`). This activates Python's internal *name mangling* mechanism, forcing programs to read or write to those fields only through controlled public methods:

```

class BankAccount:
    def __init__(self, balance):
        self.__balance = balance # Private attribute

    def get_balance(self):
        return self.__balance

account = BankAccount(1000)
# print(account.__balance) # Raises AttributeError
print(account.get_balance()) # Output: 1000

```

6. Use @property to create getter and setter methods.

The `@property` decorator allows developers to wrap getter and setter logic around private attribute access while keeping a clean, natural dot-notation interface:

```

class CelsiusTemperature:
    def __init__(self, temp):
        self._temp = temp

    @property
    def temp(self):
        return self._temp

    @temp.setter
    def temp(self, value):
        if value < -273.15:
            raise ValueError("Temperature below absolute zero is impossible.")
        self._temp = value

```

```
t = CelsiusTemperature(25)
t.temp = 30 # Triggers setter method execution implicitly
print(t.temp) # Output: 30
```

7. What is "Multiple Inheritance"? Provide a small code structure.

Multiple Inheritance lets a child class inherit attributes and methods directly from more than one parent base class:

```
class Engine:
    def start_engine(self):
        return "Engine started."

class ElectricSystem:
    def boot_system(self):
        return "Systems online."

# Inherits from both Engine and ElectricSystem base definitions
class HybridCar(Engine, ElectricSystem):
    pass

hybrid = HybridCar()
print(hybrid.start_engine()) # Output: Engine started.
print(hybrid.boot_system()) # Output: Systems online.
```

8. Create a class method using the @classmethod decorator.

Class methods are bound directly to the class blueprint instead of separate object instances. They receive the class definition context (cls) as their first parameter:

```
class Student:
    school_name = "Takshashila University"

    @classmethod
    def get_school(cls):
        return cls.school_name

print(Student.get_school()) # Output: Takshashila University
```

9. Explain the difference between a class method and a static method.

Feature Context	Class Method (@classmethod)	Static Method (@staticmethod)
	Receives the class reference identity (cls) automatically.	Receives no implicit parameters automatically (neither self nor cls).

Feature Context	Class Method (@classmethod)	Static Method (@staticmethod)
Implicit Parameter Passed		
State Modification	Can inspect or modify class-level states and configurations.	Cannot access or modify class or instance-level properties.
Primary Use Case	Factory methods initialization or global variable management.	Isolated helper utility calculations matching data logic.

10. Write a try-except-finally block to handle a ZeroDivisionError.

Error handling structures protect execution pathways by isolating risky code steps and running mandatory cleanup routines:

```
try:
    numerator = 10
    denominator = 0
    result = numerator / denominator
except ZeroDivisionError:
    print("Error caught: Cannot divide a number by zero.")
finally:
    print("Execution completed: Cleanup operations finished.")

# Output:
# Error caught: Cannot divide a number by zero.
# Execution completed: Cleanup operations finished.
```